Take-Home Assignment — Full E-Commerce Shop with Unzer Payments

Primary deliverable: An Architecture Document for the whole system, backed by a working vertical slice (checkout + Unzer payment) against the Unzer sandbox.

---

1. Context

At Unzer we build payment technology, but our merchants run online shops. To design good payment products you have to understand the system a payment lives inside. So this assignment asks you to architect a full-fledged e-commerce shop — catalog, cart, checkout, orders, inventory, customers — and integrate the real Unzer API as its payment subsystem.

This is deliberately larger than a single service: the interesting work is in how you decompose the system, keep data consistent across independently-changing parts (especially order ↔ inventory ↔ payment), and keep it operable and secure. We care far more about *how you think* — decomposition, trade-offs, consistency, failure handling — than about how many features you finish. There is no single correct answer.

You won't receive an API key up front. We'll give it to you during the technical interview so we can run and test your solution live together. You will not touch real money or real cardholder data.

> Important: Sandbox/test environment only. Never commit the API key or any secret. Use only synthetic/test data.

---

2. The System to Design

Architect the backend for a working online shop. At minimum, cover these capabilities and treat them as distinct domains — part of what we assess is where you draw the boundaries.

Customers & accounts

- Registration, login, addresses. Guest checkout allowed.
- Authentication/authorization; distinction between customer and shop-admin roles.

Product catalog

- Products, variants, prices, categories; browse and search.
- Admin management of catalog (CRUD).

Inventory / stock

- Track available stock per product/variant.
- Reserve stock at checkout and release it if payment fails or times out. Handle concurrent buyers competing for the last unit correctly (no overselling).

Cart

- Add/update/remove items; price and availability reflected at the right moments.

Checkout & orders

- Turn a cart into an order, drive it through payment, and manage the order lifecycle: `CREATED → AWAITING_PAYMENT → PAID → FULFILLING → SHIPPED → COMPLETED`, plus `CANCELLED`, `PAYMENT_FAILED`, `REFUNDED`.
- The critical hard part: keep order state, stock, and payment state consistent across parts that can each fail independently. Choosing and justifying an approach to this is central to the assignment.

Payments (Unzer integration)

- Integrate the real Unzer API for checkout. Implement all three methods below.
- Use the Unzer resource/transaction model (create a payment *type* resource, then run `authorize` / `charge`), handle redirects, and reconcile via webhooks — do not rely on the redirect return alone.
- Support refunds (full/partial) tied to order cancellation/returns.

Method	Notes from the Unzer docs
Credit Card	3-D Secure. Use UI Components / tokenization so raw card data never touches your backend (PCI scope reduction). Supports authorize and charge.
Wero	European wallet (EPI). Type `/types/wero` (prefix `wro`). Charge flow with redirect: create resource → `charge` → forward to `redirectUrl` → customer returns to your `returnUrl` → show result. Refund / partial refund supported; no authorize.
Open Banking	Bank-transfer / instant-transfer. Redirect-based charge flow, conceptually similar to Wero.

Cross-cutting requirements (system-wide)

- Consistency & idempotency — retries and duplicate webhooks must not double-charge, double-ship, or oversell.
- Reliability — any component (incl. Unzer) can be slow/down; degrade gracefully, never lose an order or a payment; reconcile webhook vs. redirect vs. polling.
- Auditability — order and payment state transitions are traceable end to end.
- Scalability — reason about where load concentrates (catalog reads, checkout writes) and how you'd scale each independently.
- Security — authn/authz, secret handling (the API key!), keeping card data out of scope.

---

3. Architecture Expectations (this is where we push you)

We expect a genuinely thought-through architecture, not a single CRUD app. Address explicitly:

- Decomposition — modular monolith vs. services? Where are the boundaries and why? What are the module/service contracts?

- Data ownership — which part owns which data; shared vs. separate databases; how the parts reference each other's data.
- Consistency strategy — how order/inventory/payment stay consistent across failures, and how you avoid lost or duplicate processing.
- Sync vs. async — which interactions are synchronous request/response and which are event-driven, and why.
- The oversell problem — your concrete mechanism for preventing overselling, and its trade-offs under concurrency.
- Failure & recovery — payment succeeds but the order-update fails; a webhook arrives before the redirect; Unzer times out mid-charge; a reservation expires after payment. Walk through how the system recovers.

Diagrams strongly encouraged and must be authored in Mermaid or PlantUML (C4 context + container, plus a sequence diagram for checkout, and a state diagram for the order lifecycle).

---

4. Getting Started with Unzer

- API & concepts: https://docs.unzer.com/online-payments/
- Direct API & managing resources: https://docs.unzer.com/server-side-integration/direct-api-integration/manage-api-resources/
- Java SDK (recommended given the role): https://docs.unzer.com/server-side-integration/java-sdk-integration/manage-java-resources/
- API reference: https://docs.unzer.com/reference/api/

Read "Payments in a nutshell" and the payment-type/transaction model first — the same create-resource-then-transact-then-webhook pattern recurs across all three methods and should shape your payment subsystem.

---

5. Deliverables

5.1 Architecture Document (primary — graded most heavily)

Self-contained and readable by someone who hasn't seen this brief. Suggested structure (adapt freely):

1. Overview & assumptions — problem in your own words; scope boundaries; what's real vs. mocked.
2. System decomposition — boundaries between the parts of the system, and why. C4 context + container diagrams.
3. Domain & data model — entities, ownership, relationships, indexes; how you model money, stock, cart, order & payment lifecycles, idempotency, and links to Unzer resource/transaction IDs. Justify DB choice(s) and shared vs. separate storage.

4. Checkout & payment flow — sequence diagram for a full checkout across all involved parts; how you wrapped the Unzer resource/transaction model behind your own API; how you abstracted the three methods so adding a fourth is cheap.
5. Consistency & failure handling — your approach to keeping order/inventory/payment consistent; the oversell mechanism; redirect-vs-webhook reconciliation; concrete failure walkthroughs.
6. Technology choices (Java) — frameworks/libraries (e.g. Spring Boot, Unzer Java SDK vs. direct HTTP, JPA, messaging) and *why*. Representative code for the trickiest part of your design.
7. Deployment & DevOps (AWS) — packaging, deployment topology, and operation on AWS: which services and why, CI/CD, config, independent scaling of read-heavy vs. write-heavy paths, and observability (logging/metrics/tracing/alerting). Deployment diagram.
8. Security & compliance — authn/authz across customer/admin, keeping card data out of scope (tokenization/UI Components), secret handling, short note on PCI-DSS scope reduction.
9. Trade-offs & next steps — decisions made, what you left out, what you'd improve with more time.

Format: the submission must be a Markdown document, with all diagrams authored as Mermaid or PlantUML (source in the Markdown, not screenshots). Maximum length: 3–5 pages (diagrams excluded). Be concise — quality over length.

5.2 Working Vertical Slice (required, supports the document)

Because the full shop can't be built in the time available, build one thing deeply: a working checkout → payment → order-confirmation slice in Java (Spring Boot recommended), integrating the Unzer sandbox for at least Credit Card and one redirect method (Wero or Open Banking); design (but you may stub) the remaining method.

- Runnable locally; README with setup instructions.
- Enough to show the flow end to end (minimal checkout page, cURL, or tests) including a webhook receiver (an ngrok/localtunnel note is fine).
- Demonstrate the consistency mechanism you designed, at least for the slice you built.

A smaller, clean, well-reasoned slice beats a large messy one. Clearly document what is real vs. stubbed.

---

6. Scope Guidance — What Matters and What Doesn't

We want to see:

- Clear architectural thinking with sensible boundaries and honest trade-offs.
- A defensible approach to keeping order, inventory, and payment consistent, including a correct solution to overselling.
- Correct handling of the Unzer resource/transaction model, redirects, and webhooks.
- A sound data model and idiomatic Java.
- A realistic, justified AWS deployment.

We do NOT expect:

- Every domain fully implemented — the document covers the whole shop; the code is one deep slice.

- A polished frontend or full admin UI.
- Real production deployment, real card data, or full PCI implementation.

---

7. Evaluation Rubric

Each area scored 1 (weak) – 5 (excellent).

Area	What we look for	Weight
Architecture	Clean decomposition & boundaries, defensible consistency strategy, correct oversell handling, redirect+webhook reconciliation, honest trade-offs	35%
Databases	Sound data model, correct modelling of money/stock/lifecycles, idempotency, mapping to Unzer IDs, consistency/transactions	25%
DevOps / AWS	Realistic topology, appropriate AWS services, CI/CD, secrets management, independent scaling, observability	20%
Java	Sound tech choices, idiomatic representative code, correct tricky part, sensible use of Unzer Java SDK / API	20%

Cross-cutting qualities throughout: clarity of communication, reasoning over feature count, security hygiene with the API key, and awareness of what you don't know.

---

8. Rules & Logistics

- Expected effort: 4–8 hours. Focused, well-reasoned work beats exhaustion — remember the code is one slice, the doc is the whole system.
- API key: you will not receive it in advance. We provide it during the technical interview and test your solution live together.
- Independence: Do this on your own. You may use docs, blogs, and AI tools as you would on the job — but you must understand and defend every decision in a follow-up. We will ask.
- Assumptions: Where ambiguous, make a reasonable assumption, state it, move on.
- Submission: the architecture document + a link to a repository (or a zip). No secrets in history.
- Follow-up: shortlisted candidates present the project in a 15–20 minute discussion (walk us through the design and the live slice), followed by other technical questions. Come ready to discuss alternatives you rejected and how you'd scale/evolve the system.
- Questions: if genuinely blocked, reach out — good questions are a positive signal.

Good luck — we're looking forward to seeing how you think.